FinSphere Nexus 2.0

Run and test guide

Start the whole system, run every business flow in the web app and in Swagger UI, and see how each failure is handled, one step at a time.



Chit funds and daily-collection loans

Java 25, Spring Boot 4.1, Spring Cloud 2025.1

11 October 2026

Contents

> **How to use this guide.** Follow steps 1 to 5 once to start the system. Steps 6 to 9 test it: the web app as an administrator and as a member, the APIs in Swagger UI, and the failure handling.
>
> Each step says what to do, what you should see, and what happens when something goes wrong. The reference sections after the steps list every API operation, every error, every time limit and every redirect.

Run and test, step by step

1. What you need
2. Configure
3. Start the whole system
4. Check that it is healthy
5. Create the first administrator
6. Run the business flow in the web app (administrator)
7. Run the member flow
8. Test the APIs in Swagger UI
9. Test the failure handling
10. Automated checks, stop, restart and reset

API reference

Auth service (port 7001, or through the gateway on 7080)

Chit and finance service (port 7002, or through the gateway on 7080)

Exception handling reference

What the web app shows

What the APIs answer

Timeouts: nothing waits forever

Loading screen

Page and route redirects

Theme and devices

Works on every screen

Change the whole theme in one place

Troubleshooting

Run on your machine for development

Run and test, step by step

Every step says what to do, what you should see, and what happens when something goes wrong. The example values (names, mobile numbers, amounts) are only examples, use your own.

1 What you need

- Docker Desktop (Windows, macOS) or Docker Engine with Compose v2 (Linux).
- At least 8 GB of memory and 4 CPU cores for Docker, and about 5 GB of free disk.
- Internet access for the first build (Docker downloads the base images and Maven dependencies).
- A browser: Chrome, Edge, Firefox or Safari, on a computer or a phone.
- `curl` for a few command-line checks (Windows: use `curl.exe` in PowerShell, or Swagger UI instead).
- These ports free on your machine:

Port	Used by
7080	Web app and APIs, through the gateway
7001	Auth service and its Swagger UI
7002	Chit and finance service and its Swagger UI
7888, 7761	Config server, service registry
5433, 7379, 7092, 17017	PostgreSQL, Redis, Kafka, MongoDB
3000, 9090, 3100	Grafana, Prometheus, Loki (only with the observability profile)

2 Configure

1. Unzip `FinSphere-Nexus-2.0.zip` and open a terminal in the `FinSphere-Nexus` folder.
2. Create your settings file:
   - Linux and macOS: `cp .env.example .env`
   - Windows PowerShell: `Copy-Item .env.example .env`
3. Open `.env` and change the secrets:
   - `FSN_JWT_SECRET`: a random text of at least 32 characters. It signs every session.
   - `POSTGRES_PASSWORD` and `FSN_DB_PASSWORD`: the database passwords.
   - `CONFIG_SERVER_PASSWORD`: the password the services use with the config server.
4. Keep `FSN_COOKIE_SECURE=false` while the site is served over `http://`. Set it to `true` once it is served over HTTPS.
5. Optional: `FSN_SUPPORT_CONTACT` is shown on the "Forgot your password?" page (for example `the Mylapore branch office, 044 2499 1234`).

Change the database passwords before the first start. The database role is created once, when the PostgreSQL volume is created, so a later change needs a reset (see step 10).

3 Start the whole system

1. Build and start everything:

   ```
   docker compose up -d --build
   ```

   - The first build compiles all modules inside Docker: 5 to 10 minutes.
   - Later starts take about a minute.
2. Docker starts the parts in the right order by itself:
   - PostgreSQL, Redis, Kafka and MongoDB first,
   - then the config server, then the service registry,
   - then the auth, chit and finance, web app and gateway services.
3. Watch the progress:

   ```
   docker compose ps
   ```

4. Wait until every service shows `healthy`. That takes 1 to 3 minutes after the build.
5. If a service is slow to become healthy, follow its log: `docker compose logs -f auth-service`

Optional logs, metrics and dashboards (Grafana on http://localhost:3000, with Loki and Prometheus connected):

```
SPRING_PROFILES_ACTIVE=loki docker compose --profile observability up -d
```

4 Check that it is healthy

1. Open http://localhost:7080. The sign-in page appears.
2. Open http://localhost:7761. The registry lists `AUTH-SERVICE`, `CHIT-FINANCE-SERVICE`, `FRONTEND-SERVICE` and `GATEWAY-SERVICE`, each with status `UP`.
3. Open http://localhost:7001/swagger-ui.html and http://localhost:7002/swagger-ui.html. Both API pages load.
4. Ask the services directly:

   ```
   curl http://localhost:7001/actuator/health      # {"status":"UP",...}
   curl http://localhost:7002/actuator/health      # {"status":"UP",...}
   curl -i http://localhost:7080/actuator/health   # 404, never public
   ```

   The last answer is meant to be 404: health and metrics are never public through the gateway.

If the sign-in page does not appear, see Troubleshooting.

5 Create the first administrator

The sign-up page creates members only. The very first administrator of a new database is created through the API, once. Do this before you share the address with anyone.

Option A, from a terminal (Linux, macOS, Git Bash):

```
curl -i http://localhost:7080/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"fullName": "Arun S", "phoneNumber": "9876543210",
       "password": "ChangeMe123", "address": "Branch office, Chennai",
       "role": "ADMIN"}'
```

- Expected: `HTTP/1.1 201`, and a body with `"success":true` and the new account in `data`.

Option B, in Swagger UI (any system):

1. Open http://localhost:7001/swagger-ui.html.
2. Open **POST /auth/register** and choose **Try it out**.
3. Paste the JSON body from option A and choose **Execute**.
4. Expected: `201`.

What can go wrong:

- `400` with `errors`: a field is not valid, for example `"phoneNumber": "Enter a 10-digit mobile number"` or `"password": "Use both letters and numbers in the password"`. Correct it and send again.
- `409`: `"This mobile number already has an account"`. The administrator already exists, sign in with it.
- `403`: `"Only an administrator can add another administrator."` An administrator already exists. Sign in as that administrator and add the new one in the web app (Members, then Add member, Account type Administrator).

6 Run the business flow in the web app (administrator)

This is the complete life of a chit scheme and a loan: members, scheme, slots, monthly auction, loan, collections.

6.1 Sign in

1. Open http://localhost:7080. You land on the sign-in page.
2. Enter the administrator's mobile number (or e-mail) and password, then choose **Sign in**.
3. While the request runs the button reads "Signing in...", and on a slow connection the loading screen shows "Signing you in...".
4. You land on the **Overview** with Running schemes, Latest payments, Latest loans and Members.

If something goes wrong:

- Wrong details: "The mobile number, e-mail or password is not correct." The form stays filled in except the password.
- Five wrong passwords in a row: "Too many failed sign-in attempts." Sign-in for that account pauses for 15 minutes.
- Empty fields: "Enter your mobile number or e-mail and your password."
- Auth service down: "Signing in is not available right now. Try again in a minute."
- The sign-in page was open across a browser restart, or cookies were cleared: the page reloads with "This page had expired, so nothing was saved. Fill in the form again and send it."

6.2 Add members

1. Open **Members** and choose **Add member**.
2. Fill in Full name, Mobile number, Address, and optionally E-mail and Care of (for example `S/o Mani`).
3. Choose the Account type: **Member** or **Administrator**.
4. Type the password twice (8 to 30 characters, letters and numbers).
5. Choose **Add member**. The loading screen says "Adding the member...".
6. You return to the member list with "Member added. They appear in member lists within a few seconds."
7. Wait about 5 seconds before booking a new member into a scheme. The scheme service learns about members from the auth service through Kafka.

Members can also sign up themselves at http://localhost:7080/register ("Create an account").

If something goes wrong:

- A message appears next to each field that needs changing, for example "Enter a 10-digit mobile number". Nothing is saved, and the form keeps what you typed (except the passwords).
- "The two passwords do not match".
- "This mobile number already has an account" or "This e-mail address already has an account".

6.3 Create a scheme

1. Open **Schemes** and choose **New scheme**.
2. Fill in the form. Example:

Field	Example
Scheme name	Gold 1 lakh, January batch
Chit value	100000
Monthly installment per member	5000
Number of members (slots)	20
Duration	20 months
Organiser commission	5 %
Highest bid allowed	40000
First month starts on	the 1st of next month
Payment window	10 days

3. The preview under the form shows the scheme's figures as you type.
4. Choose **Create scheme**. The loading screen says "Creating the scheme and its monthly schedule...".
5. The new scheme's page opens with "Scheme created with its monthly schedule." Its Monthly ledger lists all 20 months with their payment windows.

If something goes wrong:

- Missing or out-of-range values: a message next to the field, for example "Enter a scheme name" or "A scheme needs at least 2 members".
- A value that is not a number, for example `1 lakh`: the message reads `"1 lakh" is not a valid value here`.

- "The bid limit cannot be more than the chit value".

6.4 Book slots

1. On the scheme page, the **Slots** board shows every slot as free, taken or won.
2. In **Book a slot**, choose the Member, then tap a free slot on the board (or pick it in the Slot list).
3. Choose **Book slot**. The loading screen says "Booking the slot...".
4. The page reloads at the Slots board with "Slot booked." One member can hold more than one slot.
5. Book a few members (at least one) before the first auction.

If something goes wrong:

- "That slot is already taken." Someone booked it a moment earlier. The board shows the current state, pick another free slot.
- "No active member with id ... New registrations appear here within a few seconds." next to the member: the member was added a moment ago. Wait a few seconds and book again.
- "That slot is not part of this scheme." or "This scheme is closed to new members."

6.5 Settle the month's auction

1. On the scheme page, find **Settle an auction** (administrators only, while a month is open).
2. Choose the Month (the open month is preselected), the Winner (members of this scheme who have not yet won for every slot they hold) and enter the Winning bid, for example 15000.
3. The preview shows what the winner receives, the dividend per member and what everyone pays this month.
    - With the example scheme and a bid of 15,000: the commission is 5,000, the winner receives 85,000, the dividend is 500 per member, and each member pays 4,500 this month.
4. Choose **Settle auction**. The loading screen says "Settling the auction...".
5. The page reloads at the Monthly ledger with "Auction settled and the month is closed." The month shows the winner, the bid and the amount payable.

If something goes wrong:

- "The bid is above the scheme's limit." with the limit next to the bid.
- "This month is already settled." Another administrator settled it first. The ledger shows the result.
- "That member is not in this scheme." or "This member has already won for every slot they hold."

6.6 Disburse a loan

1. Open **Loans** and choose **Disburse loan**, or choose **Disburse loan** on a member's row under **Members**.

2. Fill in:

Field	Example
Borrower	Priya Raman
Loan amount	10000
Interest taken up front	1000
Number of collections	100
First collection	tomorrow
Collection days	Sundays off (or Every day, Weekends off)

3. The preview shows Hand over (9,000), Collect each day (100) and the Last collection date, skipping the days off.
4. Choose **Disburse loan**. The loading screen says "Disbursing the loan...".
5. The Loans list opens with "Loan disbursed."

If something goes wrong:

- "Interest must be less than the loan amount".
- A date in the wrong format, or a value that is not a number: "... is not a valid value here".
- "No active member with id ..." next to the borrower: a member added a moment ago, wait a few seconds.

6.7 Record payments

Recording a payment takes two steps, so the list of schemes and loans always belongs to the right member.

1. Open **Payments** and choose **Record payment**.
2. Choose the **Member**. The page reloads by itself ("Loading their schemes and loans...") and lists only that member's schemes and active loans. Without JavaScript, choose **Show their schemes and loans**.
3. Choose **Payment for**: `Chit: <scheme name>` or `Loan <number>: <amount>, <daily installment> a day`.
4. Enter the Amount received, the date it was Paid on (today or earlier), How it was paid (Cash, UPI or Bank transfer), who Collected it, and an optional Note.
5. Choose **Record payment**. The loading screen says "Recording the payment...".
6. The Payments list opens with "Payment recorded."

If something goes wrong:

- "This payment does not match the member's schemes or loans." The scheme or loan does not belong to that member.
- "Choose what this payment is for", "The payment date cannot be in the future", "The amount must be more than zero".

6.8 Check the results

1. **Overview** shows the running schemes, the latest payments and loans, and the member count.
2. **Schemes**, **Loans**, **Payments** and **Members** list everything, 20 rows per page, with Previous and Next.
3. **Your account** shows your profile.

4. Every page works on a phone: the menu moves into a drawer (the button at the top left), tables turn into cards.
5. The theme follows the device (light or dark). The moon or sun button switches it and remembers the choice.

6.9 Sign out

1. Choose **Sign out** at the bottom of the menu. The loading screen says "Signing you out...".
2. The sign-in page opens with "You have signed out." The session ends in the auth service as well.
3. Going Back in the browser does not show the signed-in pages again: they are never cached, so the browser asks for them again and lands on the sign-in page.

7 Run the member flow

1. Open http://localhost:7080/register, or choose **Create an account** on the sign-in page. Fill in the form and choose **Create account** (the loading screen says "Creating your account...").
2. The sign-in page opens with "Account created. Sign in with your mobile number and password."
3. Sign in. The member's **Overview** shows Your slots, Your loans, Your latest payments and Schemes you can join.
4. Open a scheme under Schemes you can join, tap a free slot and choose **Book slot**. Members always book for themselves.
5. The scheme page shows Your slots here, and the ledger shows the months. Other members' names stay hidden.
6. **Loans** and **Payments** show only the member's own records.
7. Members cannot open administrator pages. Opening http://localhost:7080/members shows "Your account cannot open this page" (403).

8 Test the APIs in Swagger UI

8.1 Where the API pages are

Service	Swagger UI	OpenAPI document
Auth	http://localhost:7001/swagger-ui.html	http://localhost:7001/v3/api-docs
Chit and finance	http://localhost:7002/swagger-ui.html	http://localhost:7002/v3/api-docs

- The same APIs are also served through the gateway on port 7080 (under `/auth`, `/chits` and `/finance`), but the Swagger pages are only on the service ports.
- Every operation lists its possible answers: the success answer, the standard errors that can happen there (from 400, 401, 403, 404, 409, 500 and 503) and the operation's own ones (for example 429 on sign-in). Every error uses the `ApiError` schema, with an example.
- Operations without a lock icon are public: register, login, logout and validate.

8.2 How signing in works in Swagger UI

- The session is an HttpOnly cookie named `fsn_auth_token`. There is nothing to copy into **Authorize**.
- Run **POST /auth/login** on the auth page once. The browser keeps the cookie for `localhost`.

- The chit and finance page on port 7002 sends the same cookie, because cookies are shared by all ports of a host.
- Use `localhost` in both addresses. Mixing `localhost` and `127.0.0.1` gives two different hosts and no cookie.
- To act as another person, run **POST /auth/logout**, then sign in as them.

8.3 The administrator's flow in Swagger UI

Send each body with **Try it out**, then **Execute**. Note the ids in the answers, the next steps use them.

1. **Sign in** (auth page, POST /auth/login):

   ```
   { "identifier": "9876543210", "password": "ChangeMe123" }
   ```

   Expected `200`, with `userId`, `fullName` and `role` in `data`. Then **GET /auth/me** answers `200` with the full profile.

2. **Add a member** (auth page, POST /auth/register):

   ```
   { "fullName": "Priya Raman", "phoneNumber": "9876501234",
     "password": "Priya12345", "address": "Mylapore, Chennai",
     "careOf": "W/o Raman", "role": "CUSTOMER" }
   ```

   Expected `201`. Note `data.id`, the member's user id (2 in a new database).

3. **Create a scheme** (chit page, POST /chits/plans/create):

   ```
   { "name": "Gold 1 lakh, January batch", "totalValue": 100000,
     "monthlyInstallment": 5000, "totalMembers": 20, "durationMonths": 20,
     "commissionPercentage": 5, "maxBidLimit": 40000,
     "startDate": "2026-11-01", "paymentDuration": 10 }
   ```

   Expected `201`. Note `data.id`, the scheme id.

4. **Check that the member arrived** (GET /chits/members): the new member is listed after a few seconds.

5. **Book a slot** (POST /chits/enrollments/join):

   ```
   { "planId": 1, "userId": 2, "slotNumber": 1 }
   ```

   Expected `201`. Send it again: `409` "That slot is already taken."

6. **Read the scheme** (GET /chits/plans/{planId}): the 20 months of the ledger and the taken slots.

7. **Settle month 1** (POST /chits/cycles/auction):

   ```
   { "planId": 1, "monthlyCount": 1, "bidAmount": 15000, "winnerUserId": 2 }
   ```

   Expected `200`: `prizeMoney` 85000, `organiserCommission` 5000, and in `cycle` month 1 with `closed` true, `dividendPerMember` 500 and `monthlyPayableAmount` 4500. Send it again: `409` "This month is already settled."

8. **Disburse a loan** (POST /finance/loans/disburse):

```
{ "userId": 2, "principalAmount": 10000, "interestDeduction": 1000,
  "tenureDays": 100, "startDate": "2026-10-12", "leaveType": "SUNDAY" }
```

   Expected `201`: `disbursedAmount` 9000, `dailyInstallment` 100 and the `endDate` with Sundays skipped. Note the loan id.

9. **Record a payment** (POST /finance/transactions/log):

```
{ "userId": 2, "amount": 4500, "transactionType": "CHIT_PAYMENT",
  "targetId": 1, "paymentMode": "CASH", "receivedByInfo": "Office counter",
  "remarks": "Month 1", "paymentDate": "2026-10-10" }
```

   Expected `201`. For a loan payment use `"transactionType": "LOAN_PAYMENT"` and the loan id as `targetId`.

10. **Read the lists**: GET /chits/plans/active, GET /finance/loans, GET /finance/transactions (`page` counts from 0, `size` is at most 100).
11. **Sign out** (auth page, POST /auth/logout): `200`. Now any chit or finance operation answers `401`.

8.4 The member's flow in Swagger UI

1. Sign in as the member (POST /auth/login with `9876501234` and `Priya12345`).
2. GET /chits/enrollments/me, GET /finance/loans/me and GET /finance/transactions/me: `200`, only the member's own records.
3. POST /chits/enrollments/join with the member's own `userId` and a free slot: `201`.
4. The same with another member's `userId`: `403` "You can only enrol yourself into a scheme".
5. GET /finance/loans (everyone's loans): `403`. GET /finance/transactions/history/{userId} of another member: `403`.

9 Test the failure handling

Each line says what to do and what you should get. Section Exception handling reference lists every case.

9.1 In Swagger UI or curl

Do this	Expected answer
Call GET /chits/plans/active in a private window (no cookie)	`401` "Authentication required"
POST /chits/plans/create with `{}`	`400` "Some details need changing. Check the highlighted fields." and one entry in `errors` per field
Send a body that is not JSON, for example `{ "name":`	`400` "Malformed or unreadable request body"
GET /chits/plans/abc	`400`, `errors.planId` "Invalid value"
GET /chits/plans/9999	`404` "Scheme not found"
Book the same slot twice	`409` "That slot is already taken."
Settle a month with a bid above `maxBidLimit`	`400` "The bid is above the scheme's limit."
Record a payment for a scheme the member is not in	`400` with `errors.targetId`
Sign in with a wrong password 5 times, then once more	`401` five times, then `429` "Too many failed sign-in attempts." for 15 minutes
Register a second ADMIN without an administrator session	`403` "Only an administrator can add another administrator."

Every error answer carries `correlationId`. Search the logs for it to see the whole request across the services.

9.2 With a part of the system stopped

Stop one part, try, then start it again with `docker compose start <name>`.

Stop	Then	Expected
`docker compose stop mongo`	GET /finance/transactions (port 7002)	`503` "A backing service is temporarily unavailable. Please try again shortly" within about 5 seconds
`docker compose stop redis`	Sign in	`503` within about 2 seconds, the web app says "Signing in is not available right now."
`docker compose stop postgres`	Open Schemes in the web app	"Part of the system is not responding" (503) within about 5 seconds
`docker compose stop chit-finance-service`	Open the Overview	The page still opens, and each section says "We could not reach the schemes and payments service to load ..."
Open Record payment and fill it in, then `docker compose stop chit-finance-service`	Send the form	"We could not reach the schemes and payments service, so nothing was saved." (503)
`docker compose stop auth-service`	Sign in	"Signing in is not available right now. Try again in a minute."

No request hangs: every wait has a limit (see Timeouts).

9.3 In the browser

Do this	Expected
Open http://localhost:7080/schemes/9999	"We could not find that page" (404)
Open http://localhost:7080/members?page=99	The last page of the member list
Open http://localhost:7080/schemes/	Redirected to `/schemes`
Sign in as a member, open http://localhost:7080/payments/new	"Your account cannot open this page" (403)
Open a form, delete the `XSRF-TOKEN` cookie (developer tools, Application, Cookies), send the form	Back to the same page: "This page had expired, so nothing was saved."
Sign out in one tab, then send a form in another tab	Sign-in page with "Your session ended." After signing in you return to that form's page, nothing was saved
Open two pages with forms in two tabs, send both	Both work
Slow page (for example with a stopped database)	The loading screen, after 8 seconds a hint, after 20 seconds **Stop waiting**

10 Automated checks, stop, restart and reset

Automated checks:

- Everything at once (needs JDK 25, downloads Maven on first use): `scripts/verify.sh`
  - builds every module, runs all the tests (unit tests, the web app flow tests, the client contract tests) and validates `docker-compose.yml`.
- End to end against the running stack: `scripts/smoke-test.sh` (Windows: `java scripts\SmokeTest.java`)
  - creates the first administrator on a fresh database (mobile `9000000001`, password `Admin12345`), adds a member, creates a scheme, books slots, settles an auction, records a payment, issues a loan, checks the web pages, the CSRF protection and the role rules, and signs out.
  - On a database that already has an administrator, pass its details: `FSN_SMOKE_ADMIN_PHONE=9876543210 FSN_SMOKE_ADMIN_PASSWORD=ChangeMe123 scripts/smoke-test.sh`

Stop, restart and reset:

Goal	Command
Stop everything, keep the data	`docker compose stop` (or `docker compose down`)
Start again	`docker compose up -d`
Restart one service	`docker compose restart chit-finance-service`
Rebuild one service after a code change	`docker compose up -d --build frontend-service`
See the logs of one service	`docker compose logs -f --tail 200 chit-finance-service`
Read the log files	`docker compose exec auth-service tail -n 200 logs/auth-service-error.log`
Start from an empty database (deletes all data)	`docker compose down -v`, then `docker compose up -d`

API reference

All answers use one envelope: `success`, `status`, `message`, `data` on success, `errors` (field name to message) on failure, `path`, `correlationId` and `timestamp`. Lists are pages: `content`, `page` (from `0`), `size`, `totalElements`, `totalPages`, `first`, `last`.

Who may call:

- **Public**: no session needed.
- **Signed in**: any account with a session cookie.
- **Admin**: an account with the ADMIN role.
- **Admin or self**: administrators, or the member whose `userId` is in the path.

Auth service (port 7001, or through the gateway on 7080)

Method and path	Who	What it does	Success	Errors besides 400 and 503
POST /auth/register	Public (ADMIN role: admin, except the first)	Creates an account	201	403 second admin, 409 mobile or e-mail taken
POST /auth/login	Public	Signs in and sets the `fsn_auth_token` cookie	200	401 wrong details, 403 disabled, 409 signed in elsewhere (REJECT policy), 429 locked
POST /auth/logout	Public	Ends the session and clears the cookie	200 always	none
GET /auth/validate	Public	Checks the cookie's session is live	200	401
GET /auth/me	Signed in	The signed-in profile	200	401, 404 account removed

Chit and finance service (port 7002, or through the gateway on 7080)

Method and path	Who	What it does	Success	Errors besides 400, 401 and 503
POST /chits/plans/create	Admin	Creates a scheme and its monthly ledger	201	403
GET /chits/plans/active	Signed in	Running schemes, newest first (`page`, `size`, `sort`)	200	none
GET /chits/plans/{planId}	Signed in	A scheme with its ledger and slots	200	404
POST /chits/enrollments/join	Signed in (members: themselves)	Books a slot	201	403, 404 scheme or member, 409 slot taken or scheme closed
GET /chits/enrollments/me	Signed in	My slots	200	none

Method and path	Who	What it does	Success	Errors besides 400, 401 and 503
GET /chits/enrollments/user/{userId}	Admin or self	A member's slots	200	403
GET /chits/enrollments/plan/{planId}	Admin	A scheme's slot holders	200	403
POST /chits/cycles/auction	Admin	Settles a month's auction	200	403, 404 month, 409 already settled or winner has won for every slot
GET /chits/members	Admin	Active members known to this service	200	403
POST /finance/loans/disburse	Admin	Disburses a daily-collection loan	201	403, 404 member
GET /finance/loans	Admin	All loans, newest first	200	403
GET /finance/loans/me	Signed in	My loans	200	none
GET /finance/loans/user/{userId}	Admin or self	A member's loans	200	403
POST /finance/transactions/log	Admin	Records a payment	201	403, 404 member
GET /finance/transactions	Admin	Latest payments of everyone	200	403
GET /finance/transactions/me	Signed in	My payments	200	none
GET /finance/transactions/history/{userId}	Admin or self	A member's payments	200	403

Health for monitoring, on the service ports only: `GET /actuator/health` (public), `/actuator/prometheus` (public), everything else under `/actuator` needs an administrator.

Exception handling reference

What the web app shows

Situation	What you see	Status	What to do
A field is missing or not valid	The form again, a message next to each field, what you typed is kept	400	Correct the fields and send again
A business rule refuses the change (slot taken, month settled, bid above the limit, payment for the wrong scheme, mobile number taken)	The form again with the service's message	400 or 409	Follow the message
The member named in a form is not known to the scheme service yet	A message next to the member field	404	Wait a few seconds, send again
Not signed in	The sign-in page, then back to the page you asked for	302	Sign in
The session ended (expired, signed out elsewhere, cookie no longer valid)	The sign-in page with "Your session ended. Sign in again to continue.", then back to the page, or to the form's page after a save	302	Sign in, then send the form again (nothing was saved)
The page's security token expired (cookies cleared, page restored after closing the browser)	The same page again with "This page had expired, so nothing was saved."	302	Fill in the form again
A record that does not exist, or an address like `/schemes/abc`	"We could not find that page"	404	Go to the overview
A member opens an administrator page	"Your account cannot open this page"	403	Ask an administrator
A service cannot be reached (stopped, restarting)	"Part of the system is not responding", naming the service. For a save: "... so nothing was saved."	503	Try again in a minute
A service is up but its database is not answering, or it took too long	"... could not finish this request: a database it needs is not answering, or it took too long."	503	Try again shortly. After a save, check the list first
A save got no answer in time	"... did not answer in time. Your change may still have been saved, so check the list before you try again."	503	Check the list, then decide
One section of the overview fails	That section shows the reason, the rest of the page shows normally	200	Refresh later
A service fails unexpectedly	"That did not work", with a reference id. Internal details are never shown	502	Quote the reference to support
The web app itself fails unexpectedly	"Something went wrong on our side", with a reference id	500	Quote the reference to support
An address that does not exist	"We could not find that page" (sign-in first when signed out)	404	Use the menu

The reference id on every error page is the correlation id. Every log line of that request, in every service, carries it as `cid=...`.

What the APIs answer

Status	When	Example `message`
400	A field is missing or not valid (`errors` names each field), the body is not JSON, a path or query value has the wrong type, a business rule (bid above the limit, slot outside the scheme, payment target)	"Some details need changing. Check the highlighted fields."
401	No session cookie, the session expired or was signed out, or wrong details at sign-in	"Authentication required"
403	The role does not allow it, a member acting for someone else, a disabled account, a second administrator without an administrator's session	"You do not have permission to perform this action"
404	No record with that id, or an unknown address	"Scheme not found"
405	Wrong HTTP method	"HTTP method GET is not supported here"
409	Duplicate mobile number or e-mail, slot taken, month settled, a record changed by someone else at the same time	"That slot is already taken."
415	The body is not `application/json`	"Content type is not supported, send application/json"
429	Five wrong passwords in a row	"Too many failed sign-in attempts."
500	Anything unexpected. Logged with the stack trace, never sent to the client	"An unexpected system error occurred"
503	PostgreSQL, Redis or MongoDB not answering, the connection pool exhausted	"A backing service is temporarily unavailable. Please try again shortly"
503	A database statement or transaction ran past its limit and was cancelled	"The request took too long and was cancelled. Check whether the change was saved before trying again"

Through the gateway (port 7080) two more answers are possible, as JSON for API calls and as a page for browsers:

- `503` "The service is temporarily unavailable. Please try again in a moment": no running instance of the service.
- `504` "The service took too long to respond. Please try again": no answer within 30 seconds.

Timeouts: nothing waits forever

Where	Limit	When it is reached
Gateway to a service	connect 3 s, answer 30 s	503 or 504
Web app to a service	connect 2 s, answer 10 s	The "Part of the system is not responding" page
Overview sections (loaded in parallel)	15 s per section	That section says it is taking too long, the others show
Database connection from the pool	5 s	503
PostgreSQL	connect 5 s, a silent dead connection 60 s	503
Database transaction	15 s (`spring.transaction.default-timeout`)	Rolled back, statements cancelled, 503
Redis	connect 2 s, command 2 s	503 (sign-in and session checks)
MongoDB	server selection 5 s, connect 5 s, socket 15 s	503
Kafka (auth outbox)	5 s to send, 10 s for the broker's acknowledgement	The message is retried on the next run, the user is never kept waiting
Browser loading screen	hint after 8 s, **Stop waiting** after 20 s	The person decides to wait or stop

Loading screen

Long operations never leave a frozen page.

- Every link and form that loads a new page starts the loading screen.
- Up to 150 ms nothing changes, so fast pages do not flicker.
- After 150 ms a thin bar runs along the top of the page.
- After 0.7 s the whole page is covered: the action's own message is shown (for example "Recording the payment..."), clicks and keys are blocked so nothing is sent twice, and screen readers announce the message.
- After 8 s: "This is taking longer than usual."
- After 20 s a **Stop waiting** button appears. It stops the request and gives the page back. After a save it reminds you to check the list before sending again, so nothing is saved twice.
- The button that was pressed shows its busy label (for example "Recording...") and every form can be sent only once.
- The browser's Back button never brings back a stuck loading screen.
- With reduced motion turned on in the device settings, the screen does not animate.
- Not shown for links that open in a new tab, downloads, links to other sites, links within the same page, and forms the browser refused because a required field is empty.
- Without JavaScript the pages work as normal, just without the screen.

Page and route redirects

All redirects stay on this site. After a successful form the browser shows the list or record page, so reloading it never sends the form again.

You open or do	You go to
`/` while signed out	`/login`
`/` while signed in	`/dashboard`
Any page while signed out	`/login?next=<that page>`, and back to it after signing in
A page after the session ended	`/login?notice=session-ended&next=<that page>`, the old cookie is removed
Send a form after the session ended	`/login?notice=session-ended&next=<the form's page>`, nothing saved
`/login` while signed in	The `next` page, or `/dashboard`
`/register` while signed in	`/dashboard`
Sign in with a `next` that points to another site, or to `/login`, `/logout`, `/register`, `/error`	`/dashboard`
`/logout` typed in the address bar	A "Sign out?" page with a Sign out button (signed in), or `/login`
Sign out	`/login?notice=signed-out`
Create an account	`/login?notice=registered`
Create a scheme	`/schemes/<id>?notice=scheme-created`
Book a slot	`/schemes/<id>?notice=slot-booked#slots`
Settle an auction	`/schemes/<id>?notice=auction-settled#ledger`
Add a member	`/members?notice=member-added`
Disburse a loan	`/loans?notice=loan-disbursed`
Record a payment	`/payments?notice=payment-recorded`
A form with an error	No redirect: the same form with the messages, nothing saved
A form whose security token expired	The page the form was on, with `notice=form-expired` (signed in), or the sign-in page first when the session ended too
`/schemes/<id>/slots` or `/schemes/<id>/auction` opened directly	`/schemes/<id>#slots` or `/schemes/<id>#auction`
An address ending in `/` (for example `/schemes/`)	The same address without it (308, the query is kept)
`?page=` past the last page	The last page
A query value that is not a number (`?page=two`, `?member=abc`)	The same page without that value
`/schemes/abc`	The 404 page
A member opens an administrator page	The 403 page
An unknown `notice` in the address	Ignored, nothing from the address is shown on the page

Theme and devices

Works on every screen

The web app is one responsive layout, not a separate mobile site. It was checked for correct fit (no sideways scrolling, readable text, comfortable tap targets) on every width from a 320 px phone up to a 1920 px desktop, in both light and dark themes:

- **Phones** (320–480 px): a top bar with a slide-in menu, tables turn into stacked cards, forms and buttons go full width, the slot board wraps to fit.
- **Tablets** (600–1024 px): the same slide-in menu with wider, multi-column content.
- **Laptops and desktops** (1024 px and up): the side rail is always visible, content is capped at a comfortable reading width and centred on very wide screens.

Light or dark follows the device setting, and the moon/sun button switches it and remembers the choice. Nothing depends on an external font, script or CDN, so it renders the same offline and behind a strict Content Security Policy.

Change the whole theme in one place

All colours come from a few variables at the top of `frontend-service/src/main/resources/static/css/app.css`. To recolour the entire app — side rail, buttons, links, headings, washes, focus ring, in both light and dark — change one value, the brand hue:

```
:root {
  --brand-h: 258;    /* plum (default). try: indigo 245 · blue 212 · teal 174 · green 150 · rose
344 */
  --brand-sat: 100%; /* overall colourfulness, lower is more muted */
  --accent-h: 45;    /* the one stand-out accent (the note-yellow on a prized slot / open month)
*/
}
```

Everything else is derived from these with `hsl()`, so you never hunt through the stylesheet for colours. The whole palette for light and dark mode re-tints from the same hue. The status colours (success green, error red, caution amber) stay fixed on purpose, so a warning always reads as a warning whatever the brand colour is. After editing, rebuild the front end (or set `SPRING_THYMELEAF_CACHE=false` and refresh while developing).

Troubleshooting

Symptom	Likely cause	Fix
A container keeps restarting or stays unhealthy	Too little memory for Docker, or it is waiting for a dependency	`docker compose ps`, `docker compose logs <name>`. Give Docker 8 GB, then `docker compose up -d`
"port is already allocated"	Another program uses 7080, 5433 or another port	Stop that program, or change the left-hand port in `docker-compose.yml`
The first build fails while downloading	No internet, or a proxy	Set the proxy in Docker's settings and run the build again
Signing in works, but every page goes back to the sign-in page	The session cookie is `Secure` while the site runs over `http://` on another address (a phone on Wi-Fi)	Set `FSN_COOKIE_SECURE=false` in `.env`, then `docker compose up -d`
Services cannot reach the database after changing `FSN_DB_PASSWORD`	The database role was created with the old password	`docker compose down -v` (deletes all data), then `docker compose up -d`
"No active member with id ..." when booking a member just added	The member reaches the scheme service through Kafka within a few seconds	Wait and try again. If it lasts, check `docker compose logs auth-service kafka`
"Only an administrator can add another administrator."	An administrator already exists	Sign in as that administrator and add the new one under Members
`429` or "Too many failed sign-in attempts."	Five wrong passwords in a row	Wait 15 minutes. To unlock every account now: `docker compose exec redis sh -c "redis-cli --scan --pattern 'fsn:login-failures:*' | xargs -r redis-cli del"`
Swagger "Try it out" answers `401` after signing in	The cookie is not sent: `localhost` and `127.0.0.1` mixed, or the browser blocks cookies	Use `http://localhost` for both Swagger pages, sign in again
"This page had expired, so nothing was saved."	Cookies were cleared, or the page was restored after closing the browser	Fill in the form again
"Part of the system is not responding"	A service is stopped or restarting	`docker compose ps`, then `docker compose restart <name>`
`503` "A backing service is temporarily unavailable"	PostgreSQL, Redis or MongoDB is down	`docker compose ps`, then `docker compose start <name>`
The loading screen offers **Stop waiting**	A service or database is very slow or down	Stop waiting, check `docker compose ps`, and check the list before sending a form again
A phone on the same Wi-Fi cannot open the site	The computer's firewall blocks port 7080	Allow port 7080, open `http://<computer's IP>:7080`

To follow one request through every service, take the reference (correlation id) from the error page or the API answer and search for it: `docker compose logs | grep <id>`, or in Grafana with the observability profile.

Run on your machine for development

Needs JDK 25. The Maven wrapper downloads Maven itself.

```
docker compose up -d postgres redis kafka mongo   # or your own installations
./mvnw install -DskipTests
```

Start the services in this order, each in its own terminal, from the project root:

```
java -jar config-server/target/config-server-2.0.0-SNAPSHOT.jar
java -jar discovery-server/target/discovery-server-2.0.0-SNAPSHOT.jar
DB_PORT=5433 java -jar auth-service/target/auth-service-2.0.0-SNAPSHOT.jar
DB_PORT=5433 java -jar chit-finance-service/target/chit-finance-service-2.0.0-SNAPSHOT.jar
java -jar frontend-service/target/frontend-service-2.0.0-SNAPSHOT.jar
java -jar gateway-service/target/gateway-service-2.0.0-SNAPSHOT.jar
```

- `DB_PORT=5433` is for the Compose database, which is published on 5433 so it does not clash with a local PostgreSQL. Leave it out to use a local PostgreSQL on 5432.
- A local database needs the same role and databases the Compose init script creates:

  ```
  CREATE ROLE fsn_user LOGIN PASSWORD 'fsn@123';
  CREATE DATABASE finsphere_auth OWNER fsn_user;
  CREATE DATABASE finsphere_chit_finance OWNER fsn_user;
  ```

- The services sign the session cookie as `Secure` by default. Browsers accept that on `http://localhost`. For any other plain-HTTP address set `FSN_COOKIE_SECURE=false`.
- While editing templates, set `SPRING_THYMELEAF_CACHE=false` on the front end.
- Then follow the guide from step 4.